

The image features a background of a classical building facade with columns and arches, likely the main building of the University of Würzburg. On the left, there is a blue-tinted area with a white logo consisting of a stylized 'U' and 'W' shape. The text 'Julius-Maximilians-' is in a smaller font above 'UNIVERSITÄT WÜRZBURG' which is in a larger, bold, white sans-serif font.

Julius-Maximilians-
**UNIVERSITÄT
WÜRZBURG**

Grundlagen der Programmierung

Kapitel 17: Advanced Topics

Prof. Dr. Samuel Kounev, Daniel Grillmeyer M.Sc.

- Alle Materialien (inkl. Vorlesungs- und Übungsfolien, Vorlesungs- und Übungsaufzeichnungen, Übungsblätter, Programmieraufgaben) in diesem Kurs werden nur für Ihre persönliche Nutzung bereitgestellt. Das Teilen oder Weitergeben der Inhalte an Dritte ist generell untersagt!
- Vorlesungsmaterialien von Prof. Dr. Detlef Seese wurden als Basis verwendet
- Unterstützung bei der technischen und inhaltlichen Gestaltung des Vorlesungsmaterials leisteten: Martin Sträßer, Daniel Grillmeyer, Jóakim v. Kistowski, Dietmar Ratz, Joachim Melcher, Roland Küstermann, Jana Weiner, Hagen Buchwald, Matthes Elstermann, Oliver Schöll, Niklas Kühl, Tobias Diederich

- **Enums**
- Container & Collections
- Generics
- Javadoc
- Innere Klassen
- Anonyme Klassen
- Exceptions



Aufgelistete Werte

- Manchmal möchte man eine Variable definieren, die nur eine gewisse Menge an aufgelisteten Werten halten soll

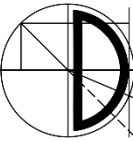
- Beispiele:

`dayOfWeek: SUNDAY, MONDAY, TUESDAY, ...`

`month: JAN, FEB, MAR, APR, ...`

`title: MR, MRS, MS, DR, PROF`

- Erinnerung: Konstanten werden in ALLCAPS geschrieben
- Was ist der eigentliche Typ dieser Konstanten?



- In der Vergangenheit wurden solche Auflistungen mit Integers realisiert:

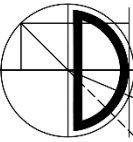
```
public final int SPRING = 0;  
public final int SUMMER = 1;  
public final int FALL = 2;  
public final int WINTER = 3;
```

- Nervig und fehleranfällig

```
int season = WINTER; ...; season = season + 1;  
int now = WINTER; ... ; month = now;
```

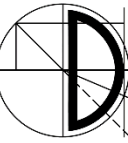
- Besser: enum-Typ nutzen

```
enum Season { WINTER, SPRING, SUMMER, FALL }
```

- Ein `enum` ist eigentlich ein neuer Klassentyp
 - Liefert Typsicherheitsprüfungen zur *Compile-Zeit*
 - Jeder deklarierte Wert ist eine eigene Instanz der enum-Klasse
 - Enums sind implizit `public`, `static` und `final`
 - Enums können mit `equals` oder `==` verglichen werden
 - Enums erben von `java.lang.Enum` und implementieren `java.lang.Comparable` (d.h., Enums sind sortierbar)
 - Enums überschreiben `toString()` and bieten `valueOf()`
 - Beispiele:

```
Season season = Season.WINTER;  
System.out.println(season);           // schreibt WINTER  
season = Season.valueOf("SPRING");    // setzt season zu Season.SPRING
```

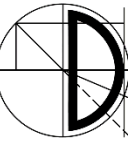


```
enum Season { SPRING, SUMMER, FALL, WINTER }  
public String germanName(Season s) {  
    switch(s) {  
        case SPRING: return "Frühling";  
        case SUMMER: return "Sommer";  
        case FALL: return "Herbst";  
        default: return "Winter";  
    }  
}
```

Aufruf: `germanName ("FALL")`

Die Methode verlangt
aber ein Objekt vom Typ
Season! Das wird bereits
zur Compile-Zeit als
Fehler erkannt

Hier wird ein String als
Parameter an die
Methode `germanName`
übergeben

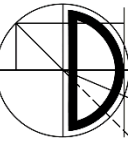


```
enum Season { SPRING, SUMMER, FALL, WINTER }  
public String germanName(Season s) {  
    switch(s) {  
        case SPRING: return "Frühling";  
        case SUMMER: return "Sommer";  
        case FALL: return "Herbst";  
        default: return "Winter";  
    }  
}
```

Aufruf: `germanName ("FALL")`

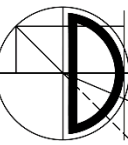
Mögliche Lösung:

`germanName (Season.valueOf ("FALL"))`

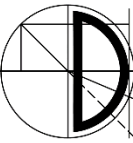


```
public void tellItLikeItIs (DayOfWeek day) {  
    switch (day) {  
        case MONDAY:  
            System.out.println("Mondays are bad.");  
            break;  
        case FRIDAY:  
            System.out.println("Fridays are better.");  
            break;  
        case SATURDAY:  
        case SUNDAY:  
            System.out.println("Weekends are best.");  
            break;  
        default:  
            System.out.println("Midweek days are so-so.");  
            break;  
    }  
}
```

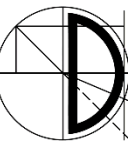
Quelle: <https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html>



- Enums bieten **Typsicherheit** zur Compile-Zeit
 - `int` enums bieten überhaupt keine Typsicherheit, z.B. `int season = 43;`
- Enums haben **lesbare Namen** für ihre Zustände
 - Bei `int` enums muss man die Zugehörigkeit der Konstanten mit Präfixen darstellen (Beispiel: `seasonWINTER` oder `S_WINTER`).
- Enums sind **robust**
 - Hinzufügen, Entfernen oder Umordnen von Konstanten erfordert Rekompilierung
- Weil Enums Objekte sind, können sie auch zu Collections (s. später) hinzugefügt werden
- Weil Enum-Typen Klassen sind, können sie auch Attribute und Methoden enthalten



- `String toString()`
 - gibt den Namen des Enum-Objektes als Konstante zurück
- `boolean equals(Object other)`
 - vergleicht die Enum-Konstanten
- `int compareTo(E o)`
 - gibt eine Ordnung zum Sortieren zurück (-1 = kleiner, 0 = gleich, 1 = größer)
- `static enum-type valueOf(String s)`
 - gibt das Enum-Objekt mit dem Namen `s` zurück
- `static enum-type[] values()`
 - gibt ein Array der Konstanten Enum-Objekte zurück



- Konstruktoren von `enums` weichen von der Standard-Darstellung ab
- Jeder aufgelistete Name im `Enum` ist ein eigener Konstruktor-Aufruf
- Beispiel:

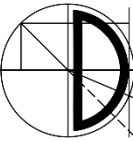
```
enum Season { WINTER, SPRING, SUMMER, FALL }
```

Erstellt 4 Objekte mit dem Standard-Konstruktor

- Beispiel 2:

```
public enum Coin {  
    private final int value;  
    Coin(int value) { this.value = value; }  
    PENNY(1), NICKEL(5), DIME(10), QUARTER(25);  
}
```

- Enum-Konstrukturen sind nur im Enum selbst verfügbar
 - Enum wird als unveränderlich angenommen



- C Enums werden ähnlich wie in Java deklariert
 - Sind aber Aliase für Integer-IDs

```
enum colors_t {black, blue, green, cyan, red, purple, yellow, white};

int main(void) {
    colors_t mycolor;
    mycolor = blue;
    if (mycolor == green) mycolor = red;
}
```

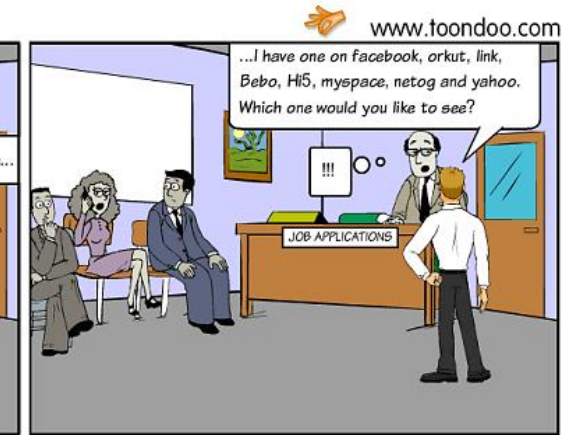
- C++ erlaubt Enum-Klassen
 - Keine Integer Aliase; Zugriff auf Werte mit Scope Resolution Operator

```
enum class Colors {black, blue, green, cyan, red, purple, yellow, white};

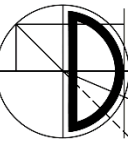
int main(void) {
    Colors mycolor;
    mycolor = Colors::blue;
    if (mycolor == Colors::green) mycolor = Colors::red;
}
```

- Enums
- **Container & Collections**
- Generics
- Javadoc
- Innere Klassen
- Anonyme Klassen
- Exceptions

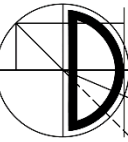
HIGH PROFILED - BY MEERASAPRA



www.toondoo.com



- Arrays
 - Eigenschaften
 - Lineare Folge fester Größe
 - `length`-Attribut
 - Kann primitive Datentypen enthalten oder Referenzen auf Objekte
 - Nur Objekte eines Typs
 - Vorteile
 - Sehr effizient
 - Typüberprüfung zur Compile-Zeit
 - Nachteile
 - Feste Größe
 - Nur ein Typ (außer bei `Object[]` mit Verwendung von Hüllklassen)
 - Wenig Methoden zum Verwalten und Ändern



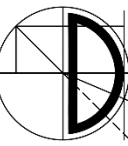
- Ein Container ist ein Objekt, das eine Sammlung von Referenzen auf andere Objekte enthält
- Ein Gruppierungsmechanismus mit komplexen Methoden zum Verwalten und Ändern der Sammlung
- Zwei unterschiedliche Konzepte von Containern

1. Collection

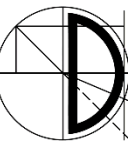
- Enthält einzelne Objekte
- z.B. in Java: Set, ArrayList, LinkedList

2. Map

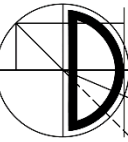
- Enthält Schlüssel-Wert Paare (key-value pairs)
- Jeder Schlüssel wird auf einen Wert abgebildet
- Jeder Schlüssel darf nur einmal vorkommen
- z.B. in Java: HashMap, Hashtable



- Eigenschaften
 - Können verschiedene Typen beinhalten
 - Keine feste Größe, automatische Größenanpassung
 - Enthalten nur Referenzen auf Objekte
 - Keine primitiven Typen (Hüllklassen können verwendet werden)
- Beispiele von Methoden
 - `add(Object o)`
 - `remove(int index)`
 - `size()`
 - `get(int index)`
 - `put(Object key, Object value) // für Map`



- Auf den folgenden Folien betrachten wir einige Beispiele zu Containern in Java
- In der Praxis werden Container de facto ausschließlich zusammen mit Generics (kommt später in diesem Kapitel) verwendet
- Die folgenden Beispiele dienen nur zur Illustration von Containern



```
import java.util.ArrayList;
```

```
// Constructs an empty list with the specified initial capacity:
```

```
ArrayList songs = new ArrayList(20);
```

```
// Add a Song:
```

```
songs.add(new Song());
```

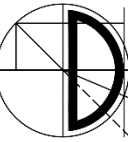
```
// Set the artist:
```

```
// (the cast is needed, to make clear it is a Song,
```

```
// you could do movies inside ☺ but ...)
```

```
((Song) songs.get(0)).setArtist("Abba");
```

ArrayList.get(i) returns a reference to an Object. The type cast (Song) is needed to tell the compiler that it is actually an instance of the Song class. This way, we can then call the method setArtist of Song.

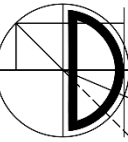


```
/**
 * This method returns a String description of all Songs
 * @return String - a String containing all Song names
 */

public String getAllSongs() {
    int i = 0;
    String result = "";

    while (i < songs.size()) {
        result = result + ((Song) songs.get(i)).toString() + "\n";
        i++;
    }

    if (i == 0)
        return ("Keine Lieder vorhanden!");
    else
        return result;
}
```



- Iteratoren: Objekte, die die folgenden zwei Methoden bieten, um einen Container zu durchlaufen

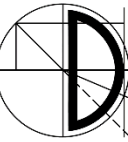
```
boolean hasNext()
```

```
// Returns true if the iteration has more elements
```

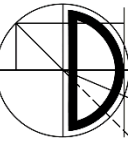
```
Object next()
```

```
// Returns the next element in the iteration
```

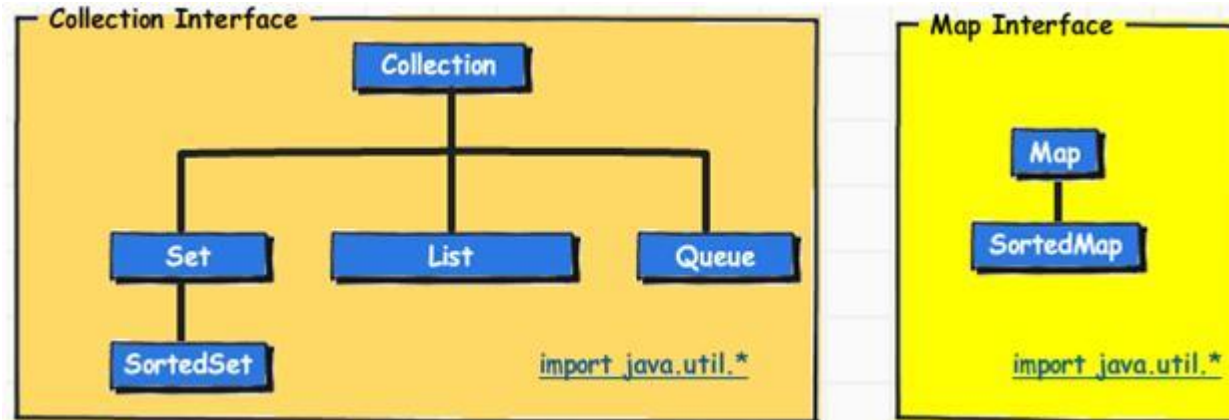
- Wozu Iteratoren?
 - Höhere Abstraktion
 - Generischer Code
 - Änderung von z.B. **ArrayList** in **LinkedList** oder **Set** möglich
- Alle drei Klassen bieten **iterator()** Methode an

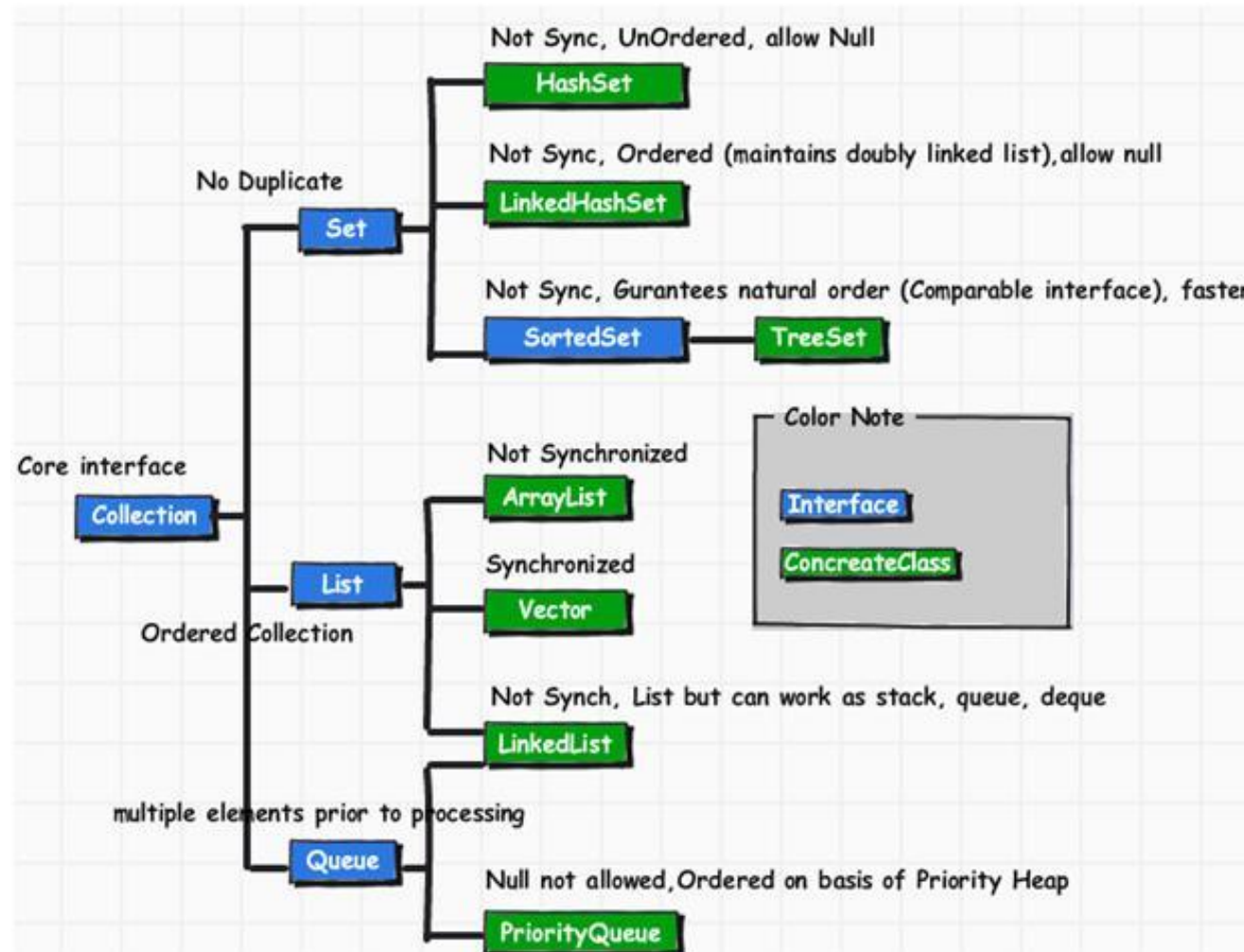
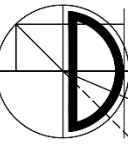


```
public String getAllSongs (ArrayList songs) {  
    String result = "";  
  
    Iterator e = songs.iterator();  
    while (e.hasNext())  
        result = result + ((Song) e.next()).toString() + "\n";  
  
    if (result.equals(""))  
        return ("Keine Lieder vorhanden");  
    else  
        return result;  
}
```

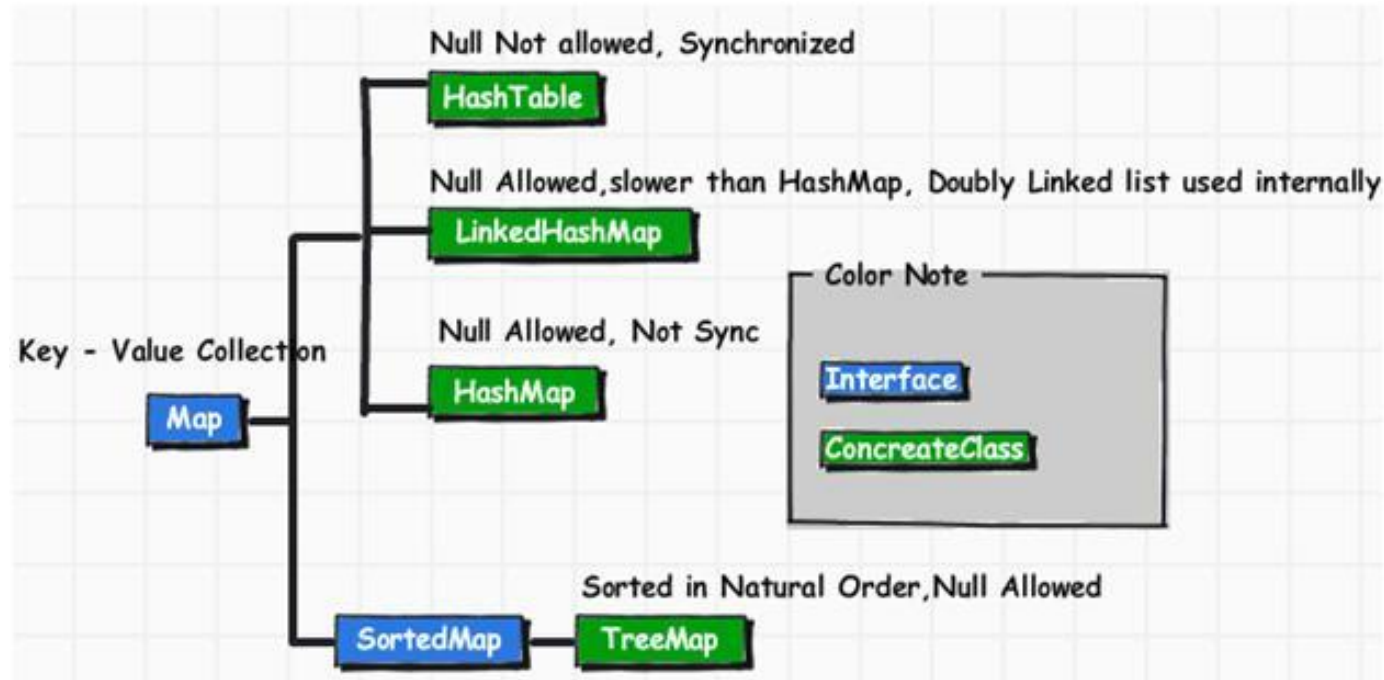
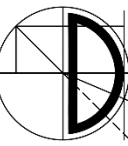



- Statt mit konkreten Klassen wird meistens mit Interfaces gearbeitet
z.B. `List songs = new ArrayList(20);`
`List` ist ein Interface in `java.util`
`ArrayList` ist eine konkrete Klasse die das `List`-Interface implementiert
`LinkedList` ist eine andere Klasse die das `List`-Interface implementiert
- Die Verwendung generischer Interfaces erleichtert den Austausch von den konkreten Klassen, z.B. von `ArrayList` zu `LinkedList`
- Wichtige Interfaces: `Collection`, `List`, `Set`, `Queue`, `Map`

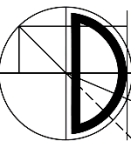




Quelle: <https://www.jitendrazaa.com/blog/java/complete-java-collection-tutorial-for-the-beginner/>



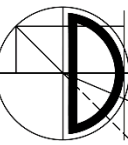
Quelle: <https://www.jitendrazaa.com/blog/java/complete-java-collection-tutorial-for-the-beginner/>



- Nachteile der Standard-Container (d.h. ohne Verwendung von Generics - s. später)
 - Weniger effizient als Arrays
 - Keine Typüberprüfung; manuelle Typumwandlungen (casts) notwendig
- Bsp. für **unterschiedliche Performance** (Anzahl für die Implementierung benötigter elementarer Operationen wie addieren, subtrahieren,...)
 - klassisches Array → Get:172; Iteration:516; Insert: na; Remove: na
 - ArrayList → Get:281; Iteration:1375; Insert:328; Remove:30484
 - LinkedList → Get:5828; Iteration:1047; Insert:109; Remove:16
- Wichtige funktionale Unterschiede

	HashSet	LinkedHashSet	TreeSet	ArrayList	Vector	LinkedList	HashTable	LinkedHashMap	HashMap	TreeMap
Null	✓	✓	✗	✓	✓	✓	✗	✓	✓	✓
Duplicate	✗	✗	✗	✓	✓	✓	✗	✗	✗	✗
Sorted Result	✗	✗	✓	✗	✗	✗	✗	✗	✗	✓
Retrieval order same as insertion	✗	✓	✗	✓	✓	✓	✗	✓	✗	✗

Quelle: <http://www.jitendrazaa.com/blog/java/complete-java-collection-tutorial-for-the-beginner/>

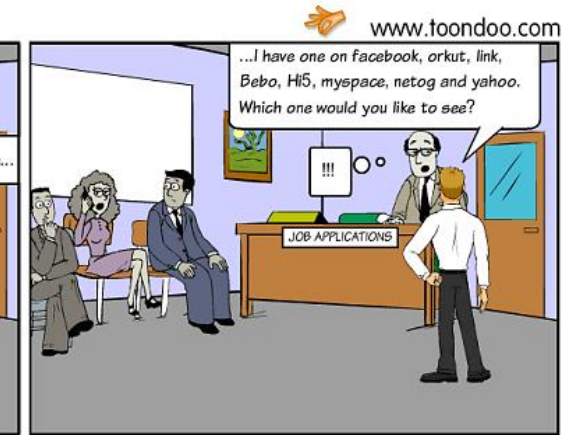


- C++ bietet viele Container-Klassen als Teil der Standard Template Library an
- Alle C++ Container sind Templates
 - Verwendung sehr ähnlich zu Java Generics (s. später)
- Beispielcontainer

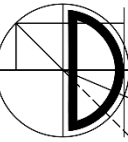
C++ Container	Java Collection	Beschreibung
Vector	ArrayList	Array mit veränderbarer Länge
List	LinkedList	Doppelt verkettete Liste
Set	SortedSet	Sortierte und iterierbare Menge in der jedes Element maximal einmal vorkommt
Map	SortedMap	Sortierte und iterierbare Tupelsammlung mit Schlüssel/Wert-Tupeln

- Enums
- Container & Collections
- Generics
- Javadoc
- Innere Klassen
- Anonyme Klassen
- Exceptions

HIGH PROFILED - BY MEERASAPRA



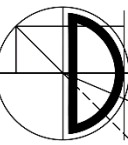
www.toondoo.com



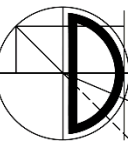
- **Hintergrund:** Alte Versionen der Java Collections (vor Java 1.5) waren *Objekt-*basiert und erforderten das Benutzen von hässlichen Typecasts
 - Typ der Elemente einer Collection kann nicht spezifiziert werden
 - Beim **Zugriff** braucht man immer **Typecast** zur spezifischen Klasse

Beispiel: `((Song) songs.get(0)).setArtist("Abba");`

- **Java Generics**
 - Ermöglichen mehr Sicherheit und Lesbarkeit
 - Vor allem für allgemeine Datenstrukturen nützlich
- **Generische Programmierung** = Programmierung von Klassen und Methoden mit parametrisierten Typen

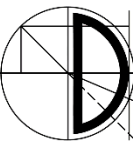


- Generische Klassen und Methoden sind ein Werkzeug zur Erstellung von wiederverwendbaren objektorientierten Typen und Bibliotheken
- Volles Verständnis erfordert Einarbeitung in Typ-Theorie
 - Insbesondere die Prinzipien der *Kovarianz* und *Kontravarianz*
- In dieser Vorlesung illustrieren wir das Prinzip hinter generischer Typisierung mit Hilfe von einigen Beispielen



- Wir wollen einen Datentyp `Paar` definieren, der zwei Objekte gleichen Typs speichern kann
- Zunächst wollen wir Objekte vom Typ `String` speichern

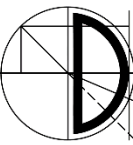
```
class StringPaar {  
    public String first;  
    public String second;  
    public StringPaar(String f, String s) {  
        first = f;  
        second = s;  
    }  
    public StringPaar(){ first = null; second = null; }  
}
```



- Nun dasselbe für Integer ...

```
class IntegerPaar {  
    public Integer first;  
    public Integer second;  
    public IntegerPaar(Integer f, Integer s) {  
        first = f;  
        second = s;  
    }  
    public IntegerPaar(){ first = null; second = null; }  
}
```

- Eigentlich haben wir an allen Stellen nur String durch Integer ersetzt – müssen wir dafür wirklich mehrere Klassen benutzen?



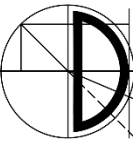
- Beispiel einer **generischen Klasse**

```
class Paar<T> {  
    public T first;  
    public T second;  
    public Paar(T f, T s) { first = f; second = s; }  
    public Paar() { first = null; second = null; }  
}
```

Die Klasse `Paar` ist
parametrisiert nach Typ `T`

- Die Klasse wird mit der Substitution des Typs `T` durch einen konkreten Typ instantiiert. Beispiel: `Paar<String>`
- Eine instantiierte generische Klasse ist wie eine ganz normale Klasse benutzbar (fast):

```
Paar<String> pair = new Paar<String>("1", "2");
```

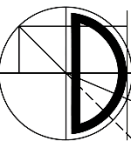


- Mehrere Typ-Parametrisierungen:

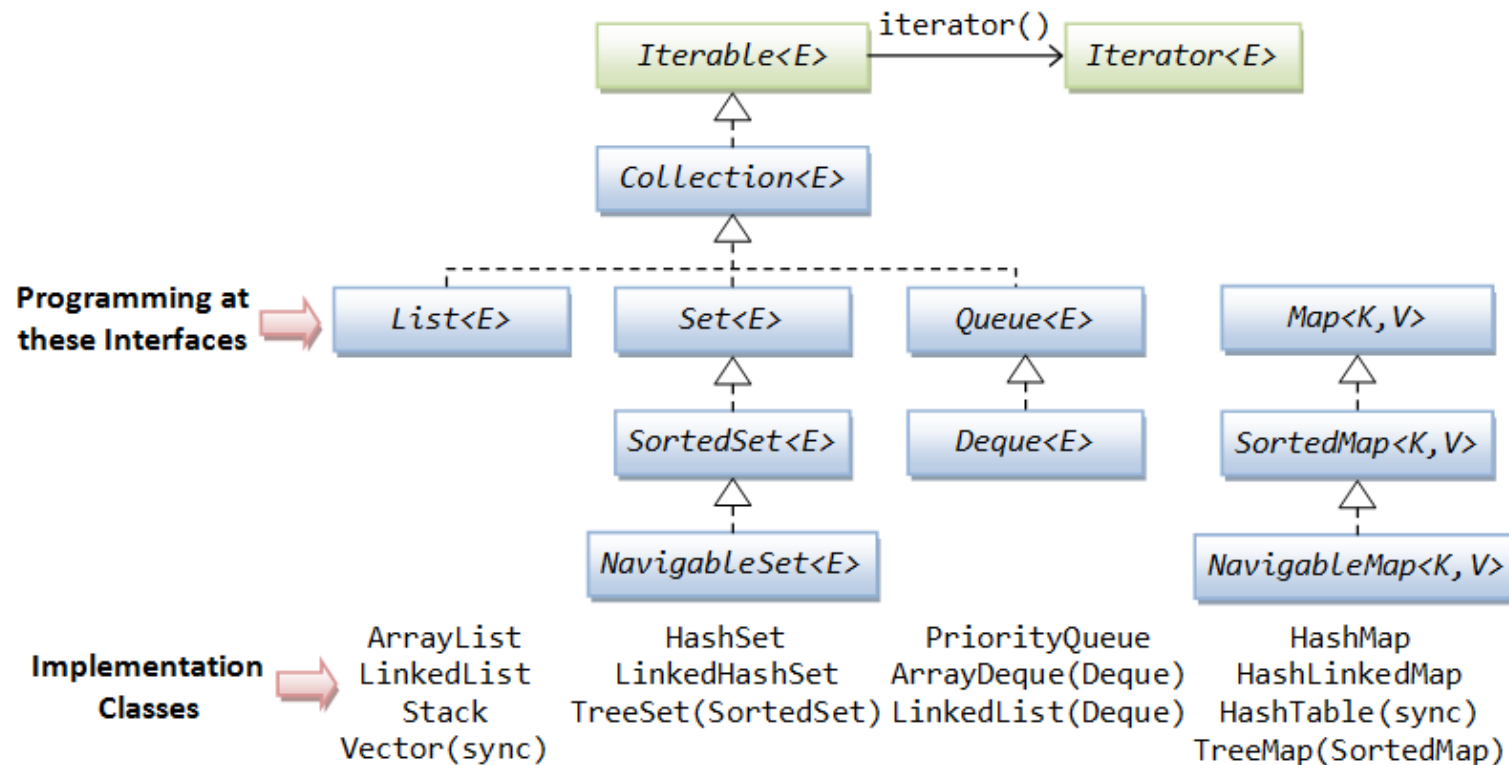
```
class Paar<T, U> {  
    public T first;  
    public U second;  
    public Paar (T x, U y) { first = x; second = y; }  
    public Paar () { first = null; second = null; }  
}
```

- Zur Instantiierung

```
Paar<String, Number>
```

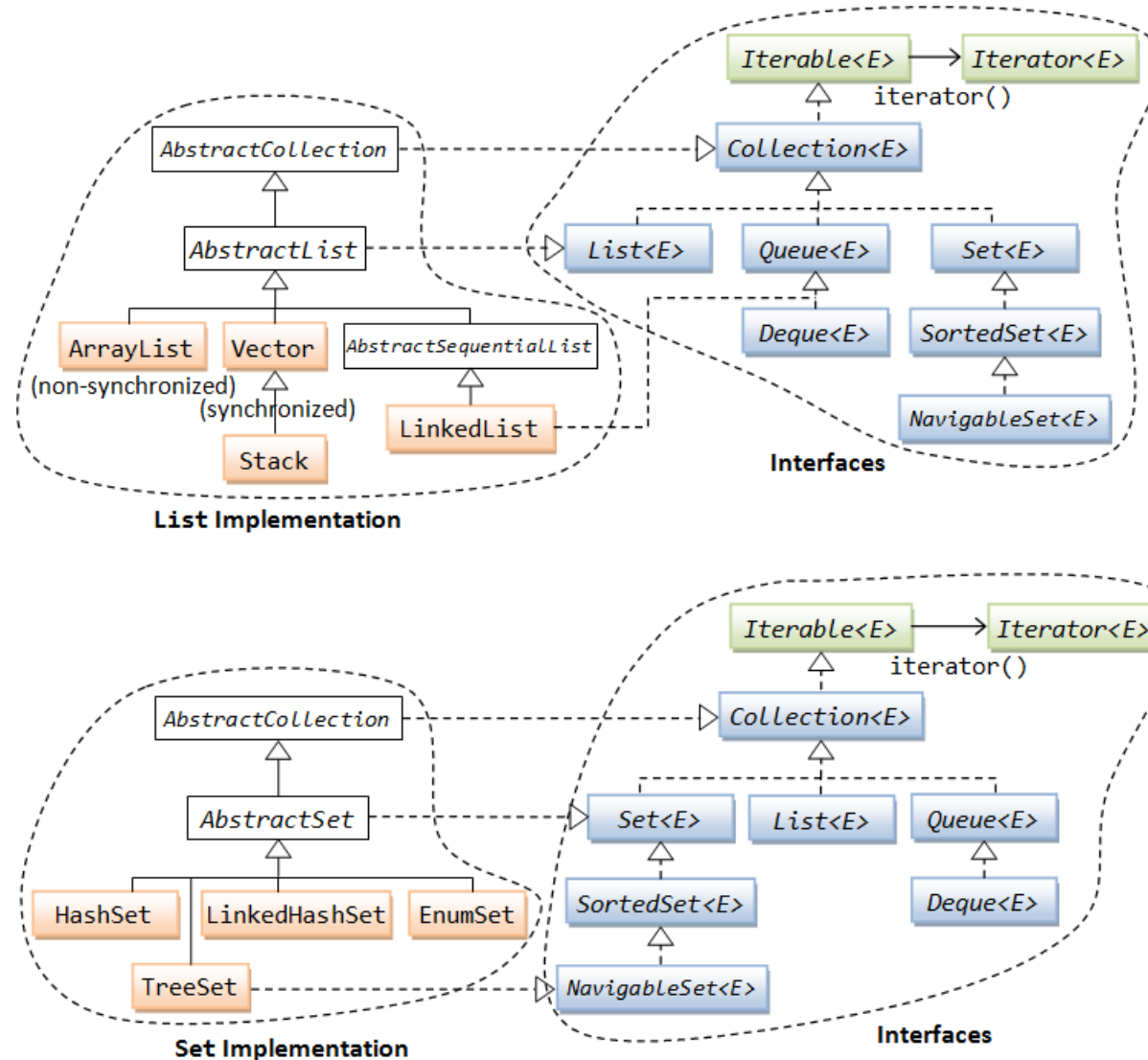
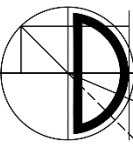


- Generics werden am häufigsten verwendet, um mit typisierten Containern zu arbeiten
- Ab Java 5 (v1.5) werden typisierte Container-Klassen unterstützt

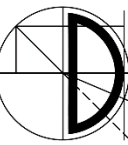


Quelle:

http://www.ntu.edu.sg/home/ehchua/programming/java/J5c_Collection.html



Quelle: http://www.ntu.edu.sg/home/ehchua/programming/java/J5c_Collection.html



```
import java.util.ArrayList;

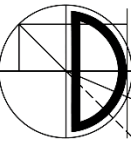
// Constructs an empty list with the specified
// initial capacity:
ArrayList<Song> songs = new ArrayList<Song>(20);

// Add a Song:
songs.add(new Song());

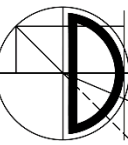
// Set the artist:
songs.get(0).setArtist("Abba");
```

songs is of type ArrayList<Song> and therefore songs.get(i) returns a reference to a Song object. No type cast is needed!





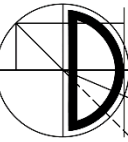
- Unterstützen *statisch typisierte* Datenstrukturen
 - *Frühe Compile-Zeit Erkennung* von Typverletzungen
 - Beispiel: Man kann keinen **String** in **ArrayList<Song>** einfügen
 - Versteckt automatisch generierte Typumwandlungen
 - Keine Typecasts notwendig
- Generische Typen sind Fabriken für *Compile-Zeit-Abkapselungen* generisierter Klassen und Methoden
- *Oberflächlich betrachtet* ähnlich wie C++ Templates



- **Generische Methoden** können in normalen und generischen Klassen definiert werden

```
class Algorithms { // irgendeine Utility-Klasse
    public static <T> T getMiddle(T[] a) {
        return a[a.length / 2];
    }
    ...
}
```

- Beim Aufruf der generischen Methode kann **der Typ** spezifiziert werden
`String s = Algorithms.<String>getMiddle(names);`
- Meist wird der Typ automatisch vom Compiler abgeleitet:
`String s = Algorithms.getMiddle(names);`



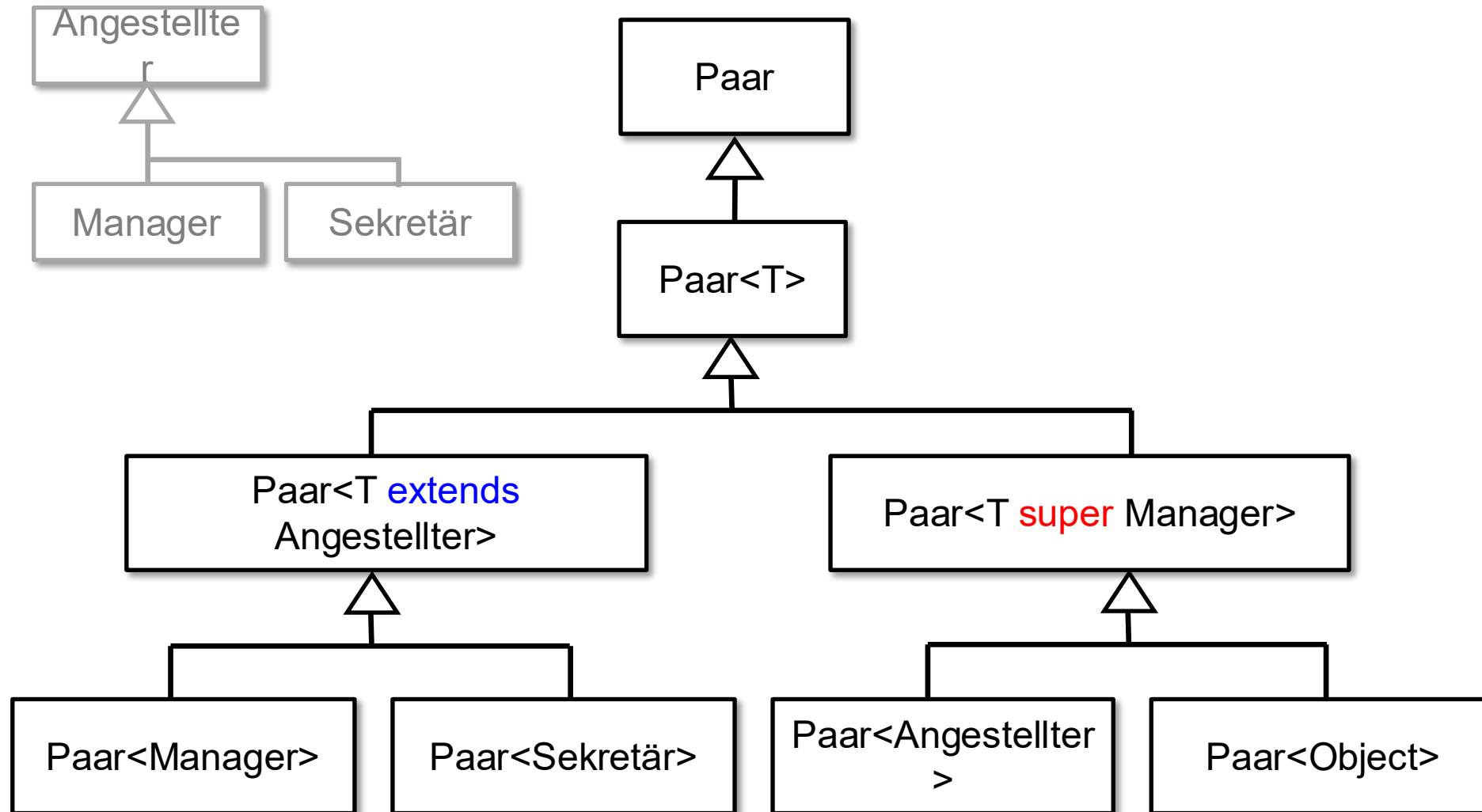
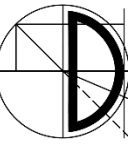
```
class Garage<T extends Vehicle> { ... }
```

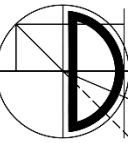
- T darf hier nur eine Subklasse von der Klasse **Vehicle** sein

```
class Car extends Vehicle {...}
```

```
Garage<Car> myGarage = new Garage<Car>();
```

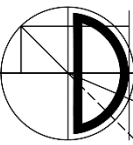
- Anstelle von **Vehicle** kann eine beliebige Klasse oder ein beliebiges Interface stehen
- Die Klasse **Garage** darf nur für einen Typ **T** instanziiert werden, der die entsprechende Klasse erweitert (Subklasse), oder im Falle von Interface, das entsprechende Interface implementiert
- Ähnlich gibt es:
Class<T super X> → T darf nur eine Superklasse von **X** sein





- `Paar<Manager>` passt auf `Paar<T extends Angestellter>` → Subtyprelation (*kovariante* Typisierung)
- `Paar<Object>` passt auf `Paar<T super Angestellter>` → Supertyprelation (*kontravariante* Typisierung)
- `Paar<Angestellter>` kann nur *Angestellte* enthalten, aber `Paar<Object>` erlaubt alles (z.B. *Numbers*) → *keine* Subtyprelation
- `Paar<T> <= Paar<?> <= Paar`

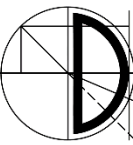
```
List<String> s1 = new LinkedList<String>();  
List x = s1;                // OK  
x.add(new Integer(5));      // Typsicherheitswarnung  
.  
.  
String str = s1.get(0);     // wirft ClassCast
```



- Auch Interfaces unterstützen generische Programmierung
- Beispiel: `java.util.Comparator`

```
public interface Comparator<T> {  
  
    int compare(T o1, T o2);  
  
}
```

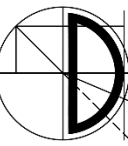
- Vergleicht zwei Objekte vom Typ T
 - Gibt eine negative Zahl zurück, wenn o1 "kleiner als" o2
 - Gibt 0 zurück, wenn o1 "gleich" o2
 - Gibt eine positive Zahl zurück, wenn o1 "größer als" o2
- Implementierung abhängig vom Typ T



- Beispiel: Vergleichen von Songs anhand ihres Erscheinungsjahres

```
class SongComparator implements Comparator<Song> {  
  
    @Override  
  
    public int compare(Song o1, Song o2) {  
  
        if (o1.getYear() < o2.getYear()) return -1;  
  
        else if (o1.getYear() == o2.getYear()) return 0;  
  
        else return 1;  
  
    }  
  
}
```

```
public interface Comparator<T> {  
    int compare(T o1, T o2);  
}
```

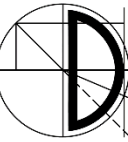


- C++ erlaubt generische Programmierung mit Klassen Templates
 - Erlauben Typ-spezifische Versionen generischer Klassen

- Format:

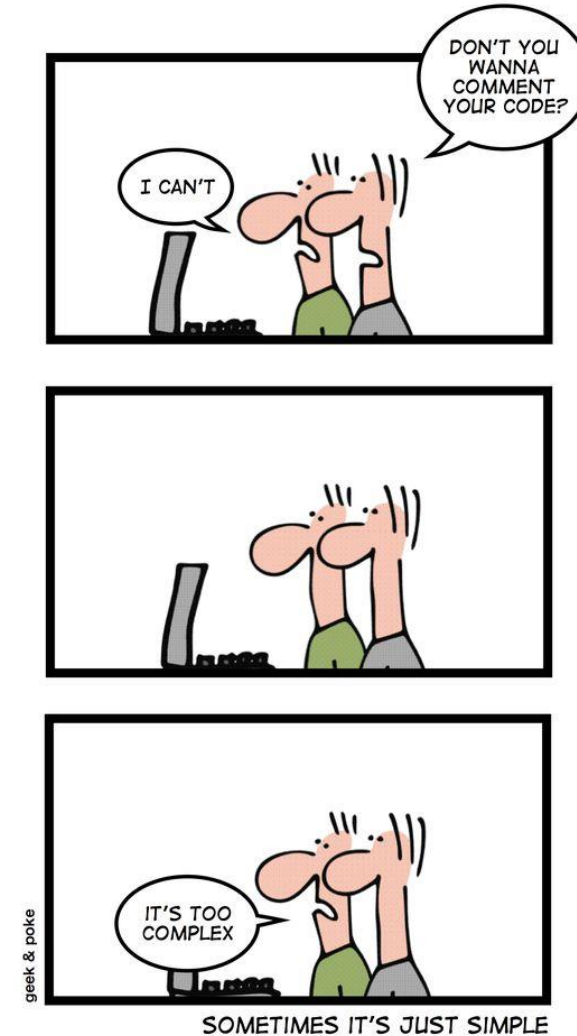
```
template <class T>
class MyClass {
    // class definition
}
```

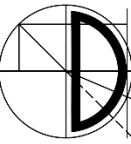
- "T" kann ein beliebiger Typ sein (z.B. `int`, `double`, `struct`)
- Objektinstanziierung:
`MyClass<T> myObject;`



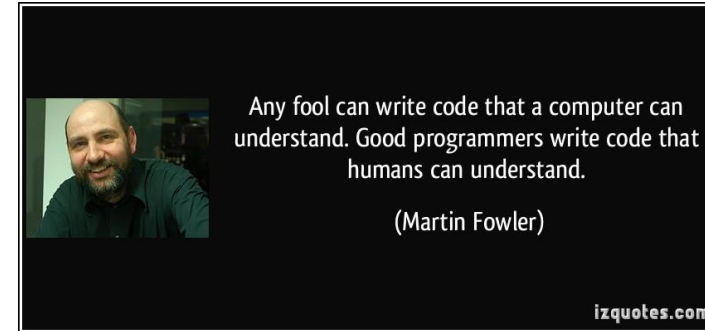
- Template Klassen-Methoden
 - Werden normal deklariert, `template <class T>` muss vor der Deklaration geschrieben werden, generische Daten in der Klasse werden mit `T` angezeigt
- Konzeptioneller Unterschied zu Java Generics:
 - Der Compiler ermittelt jede Benutzung des Templates
 - Kompiliert separate Klasse für jedes `Template<T1>, . . . , Template<Tn>`
 - ➔ Templates werden zur Compilezeit zu Klassen transformiert!
 - Java Generics werden zur Compilezeit auf Typsicherheit geprüft, aber erst zur Laufzeit zu Klassen transformiert

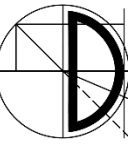
- Enums
- Container & Collections
- Generics
- Javadoc
- Innere Klassen
- Anonyme Klassen
- Exceptions





- Wozu Code kommentieren?
- Problem beim Dokumentieren
 - Aktualisierung der Dokumentation
 - Code und Dokumentation oft in separaten Dateien
- Lösung: javadoc
 - Code und Dokumentation in einer Datei
 - javadoc tags z.B. @param, @return
 - Teil der JDK Installation
 - Generiert HTML Dokumentation

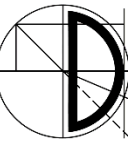




- Klassen, Methoden und Variablen
- Beispiel

```
/** A class comment */
public class MediaDataBase {
    /** A variable comment */
    public int i;

    /** A method comment */
    public void addSong(Song song) {
        ...
    }
}
```



- Beispiele

```
@author author-information
```

```
@version version-information
```

```
@param parameter-name description
```

```
@return description
```

```
@throws fully-qualified-class-name description
```

```
/**
```

```
 * This method adds a song to the List of songs
```

```
 * @param song the Song to add
```

```
 * @return void
```

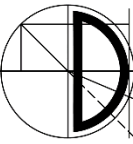
```
*/
```

```
public void addSong(Song song) {
```

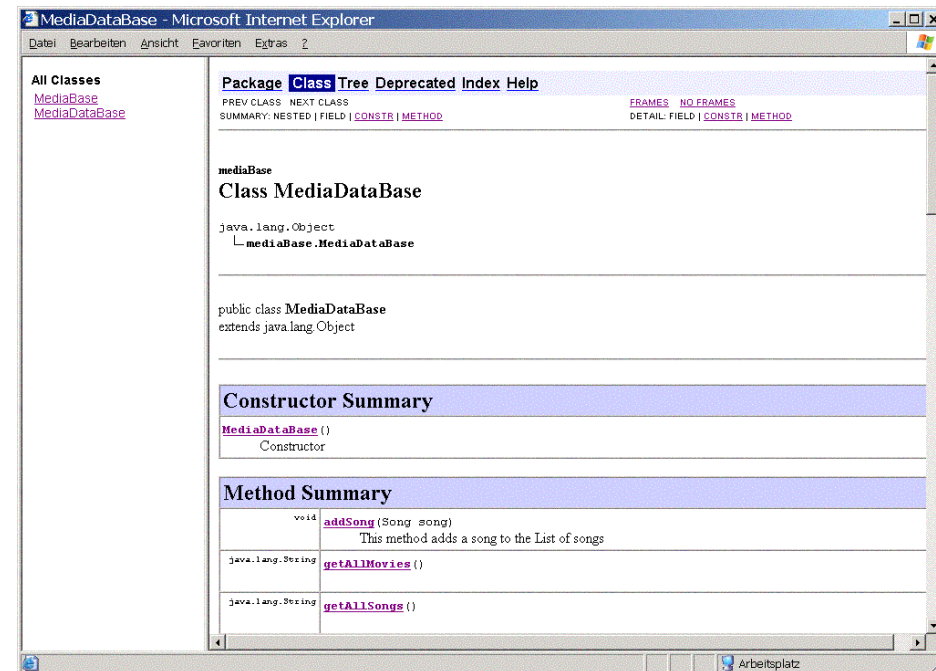
```
    int i;
```

```
    songs.add(song) ;
```

```
}
```

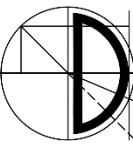


- Zum Erzeugen der HTML-Dokumentation von mediaBase:
`javac MediaDataBase -d doc mediaBase mediaBase.ui mediaBase.model`
- Siehe `javadoc -help`

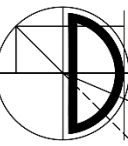


- Enums
- Container & Collections
- Generics
- Javadoc
- **Innere Klassen**
- Anonyme Klassen
- Exceptions

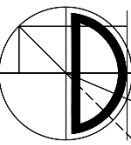




- Klassen können deklariert werden
 - innerhalb einer anderen Klasse (**geschachtelte Klasse**)
 - außerhalb einer Klasse, aber in eigenständiger Datei (**bevorzugt!**)
 - (außerhalb einer Klasse, aber in der gleichen Datei)
- Die eigenständigen Klassen/Dateien können im aktuellen Arbeitsverzeichnis oder in anderen Verzeichnissen liegen



```
public class BeispielGeschachtelteKlasse {  
    public static class Person {  
        public String  vorname;  
        public String  nachname;  
        public int     alter;  
    }  
    void main() {  
        Person otto;  
        otto = new Person();  
        otto.vorname = "Otto";  
        otto.nachname = "Meier";  
        otto.alter = 22;  
        System.out.println(otto.alter);  
    }  
}
```

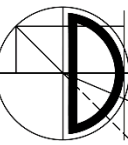



- Sondertyp von geschachtelten Klassen

```
class Außen {  
    class Innen {  
    }  
}
```

Ohne Schlüsselwort **static**!

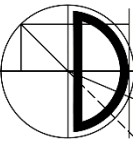
- Compiler erzeugt für **innere** Klassen eine eigene class-Datei: `Außen$Innen.class`
- Vorteil ist im wesentlichen eine reinere Objektorientierung
 - z.B. lässt sich von einer Klasse Tree ein Such-Algorithmus abspalten
- Auch Klassen innerhalb von Methoden sind nach gleichem Muster möglich
 - sog. **Lokale Klassen**



- Innere Klassen **können nur im Zusammenhang einer Instanz der äußeren Klasse** instanziiert werden
- Die Instanz der inneren Klasse ist immer mit der Instanz der entsprechenden äußeren Klasse assoziiert
- Innere Klassen können auf Komponenten der sie umfassenden Klasse zugreifen
- Eine innere Klasse ist außerhalb des sie definierenden Blockes, außer mit Hilfe des voll qualifizierenden Namens (z.B. Außen.Innen), nicht sichtbar

- Enums
- Container & Collections
- Generics
- Javadoc
- Innere Klassen
- **Anonyme Klassen**
- Exceptions





- In Java sind auch unbenannte lokale Klassen möglich, die dann **anonyme Klassen** genannt werden, diese Klassen müssen ein Interface oder eine Elternklasse implementieren
- Beispiel, wir wollen folgendes Interface inline implementieren:

```
/* MyInterface */  
public interface MyInterface {  
    // Methodensignatur  
    public int myMethod();  
}
```

```
/* main-Methode */  
void main() {  
    // Deklaration und Implementierung in  
    // Einem  
    MyInterface five = new MyInterface() {  
        public int myMethod() {  
            return 5;  
        }  
    }  
    IO.println(five.myMethod());  
}
```

Ausgabe
5

Konstruktor

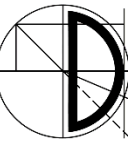
Bei Interfaces muss hier immer der Standard-Konstruktor verwendet werden. Bei Klassen der jeweilige Konstruktor der Klasse.

Geschweifte Klammer

Hier können wir Methoden überschreiben und neue Methoden einfügen.

Überschriebene Methode

Hier überschreiben wir *myMethod*. Bei Interfaces müssen alle Methoden überschrieben werden.



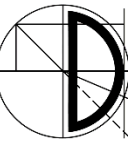
- Anonyme Klassen müssen nicht direkte Kinder des Interfaces (der Elternklasse) sein
- Beispiel: Unsere Implementierung erbt von *LinkedList*, wird aber in eine Variable vom Typ *List* geschrieben. Dies funktioniert, da *LinkedList* auch von *List* erbt.

```
class L {  
    private List list = new LinkedList() {  
        public boolean add(Object o) {  
            System.out.println("Objekt " + o + " hinzugefügt.");  
            return super.add(o);  
        }  
    };  
  
    public List getList() {  
        return list;  
    }  
}  
Aufruf: new L().getList().add("Türklinke");
```

- Die add-Methode von LinkedList wurde „nebenbei“ überschrieben, das spart also Tipparbeit, s. später auch bei Event-Adapter-Klassen

- Enums
- Container & Collections
- Generics
- Javadoc
- Innere Klassen
- Anonyme Klassen
- Exceptions





```
public class Hallo {  
    void main () {  
        System.out.println("Hallo!");  
    }  
}
```

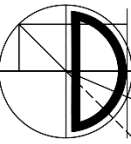
Compiliere ...
Programm compiliert.

Compilieren

Programm wird ausgeführt ...
Hallo!

Interpretieren

Programm beendet



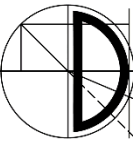
```
public class Hallo {  
    void main () {  
        System.out.println("Hallo!");  
    }  
}
```

- Compilieren

Fehlermeldungen
Tipp: Beheben Sie stets die oberste Fehlermeldung zuerst. Klicken Sie auf eine Fehlermeldung, um zur Zeile zu springen.

• Hallo.java (3,11): cannot find symbol
Methode 'println(java.lang.String)' in Klasse 'java.lang.System'

- Interpretieren
 - **Nicht möglich!**



```
public class Feld
{
    void main ()
    {
        int[][] feld = new int[5][];

        System.out.print(feld[0][0]);
    }
}
```

Compiliere ...

Programm compiliert.

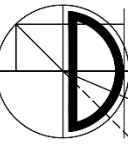
Programm wird ausgeführt ...

```
Exception in thread "main" java.lang.NullPointerException
    at Feld.main(Feld.java:9)
```

Es sind Laufzeitfehler aufgetreten.

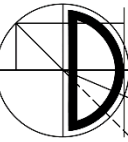
Compilieren

Interpretieren

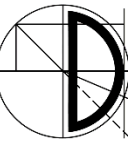


- Prinzipiell: Eine Ausnahmesituation
- Konkret in Java: Eine Instanz der Klasse `java.lang.Exception`
 - extends `Throwable`
- Auszug API:



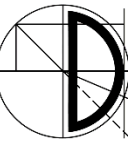


- Manchmal ist es schwer, im Vorhinein zu erkennen, welche Probleme mit den eigenen Programmen auftreten können
- Instanzen der Klasse `java.lang.Exception` helfen uns also beim Verstehen von Problemen in unserem Programm, statt es einfach „abstürzen“ zu lassen.
- Exceptions können durch den Entwickler abgefangen und damit **behandelt** werden. Dann kann das Programm wie vorgesehen weiter ausgeführt werden.
 - Dieses Behandeln von Ausnahmen nennt man **Fangen (catching)**



```
Programm wird ausgeführt ...  
Exception in thread "main"  
java.lang.NullPointerException  
    at Feld.main(Feld.java:9)
```

```
Es sind Laufzeitfehler  
aufgetreten.
```

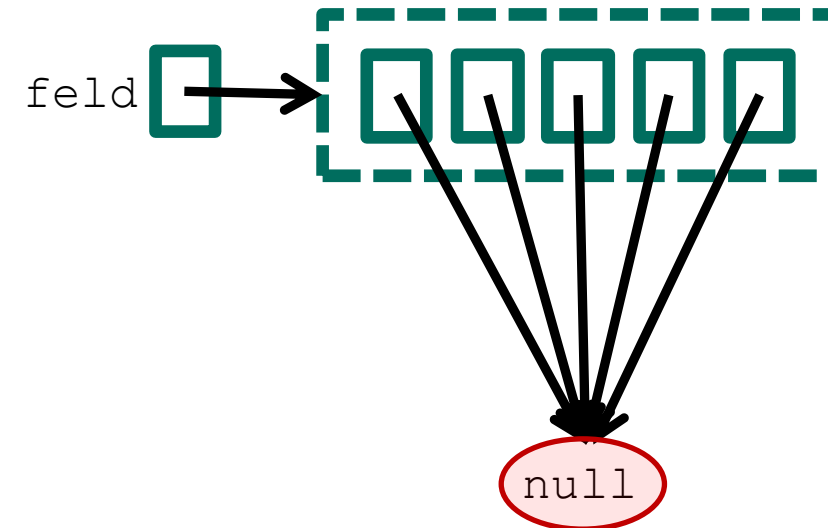


NullPointerException

Code

```
int[][] feld = new int[5][];  
IO.print(feld[0][0]);
```

Speicher

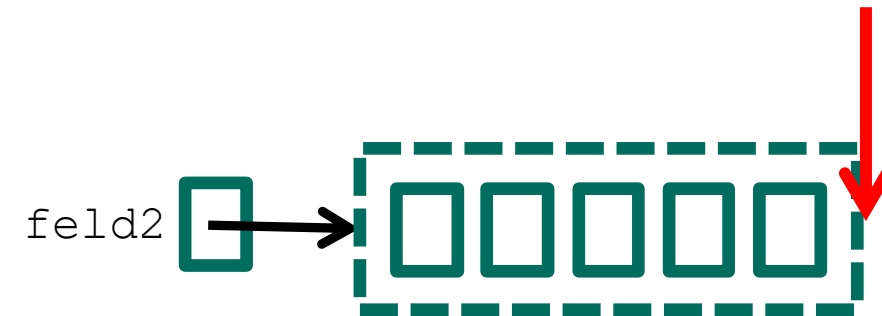


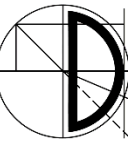
ArrayIndexOutOfBoundsException

Code

```
int[] feld2 = new int[5];  
IO.print(feld2[5]);
```

Speicher





java.lang

Class RuntimeException

[java.lang.Object](#)└ [java.lang.Throwable](#)└ [java.lang.Exception](#)└ [java.lang.RuntimeException](#)

All Implemented Interfaces:

[Serializable](#)

Direct Known Subclasses:

[AnnotationTypeMismatchException](#), [ArithmeticException](#), [ArrayStoreException](#), [BufferOverflowException](#), [BufferUnderflowException](#), [CannotRedoException](#), [CannotUndoException](#), [ClassCastException](#), [CMMException](#), [ConcurrentModificationException](#), [DataBindingException](#), [DOMException](#), [EmptyStackException](#), [EnumConstantNotPresentException](#), [EventException](#), [IllegalArgumentException](#), [IllegalMonitorStateException](#), [IllegalPathStateException](#), [IllegalStateException](#), [ImagingOpException](#), [IncompleteAnnotationException](#), [IndexOutOfBoundsException](#), [JMRuntimeException](#), [LSEException](#), [MalformedParameterizedTypeException](#), [MirroredTypeException](#), [MirroredTypesException](#), [MissingResourceException](#), [NegativeArraySizeException](#), [NoSuchElementException](#), [NoSuchMechanismException](#), [NullPointerException](#), [ProfileDataException](#), [ProviderException](#), [RasterFormatException](#), [RejectedExecutionException](#), [SecurityException](#), [SystemException](#), [TypeConstraintException](#), [TypeNotPresentException](#), [UndeclaredThrowableException](#), [UnknownAnnotationValueException](#), [UnknownElementException](#), [UnknownTypeException](#), [UnmodifiableSetException](#), [UnsupportedOperationException](#), [WebServiceException](#)

```
public class RuntimeException
extends Exception
```

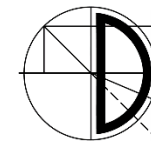
RuntimeException is the superclass of those exceptions that can be thrown during the normal operation of the Java Virtual Machine.

A method is not required to declare in its throws clause any subclasses of RuntimeException that might be thrown during the execution of the method but not caught.

Since:

JDK1.0

Quelle: <http://docs.oracle.com/javase/1.4.2/docs/api/java/lang/RuntimeException.html>



java.lang

Class Exception

[java.lang.Object](#)

└ [java.lang.Throwable](#)

└ [java.lang.Exception](#)

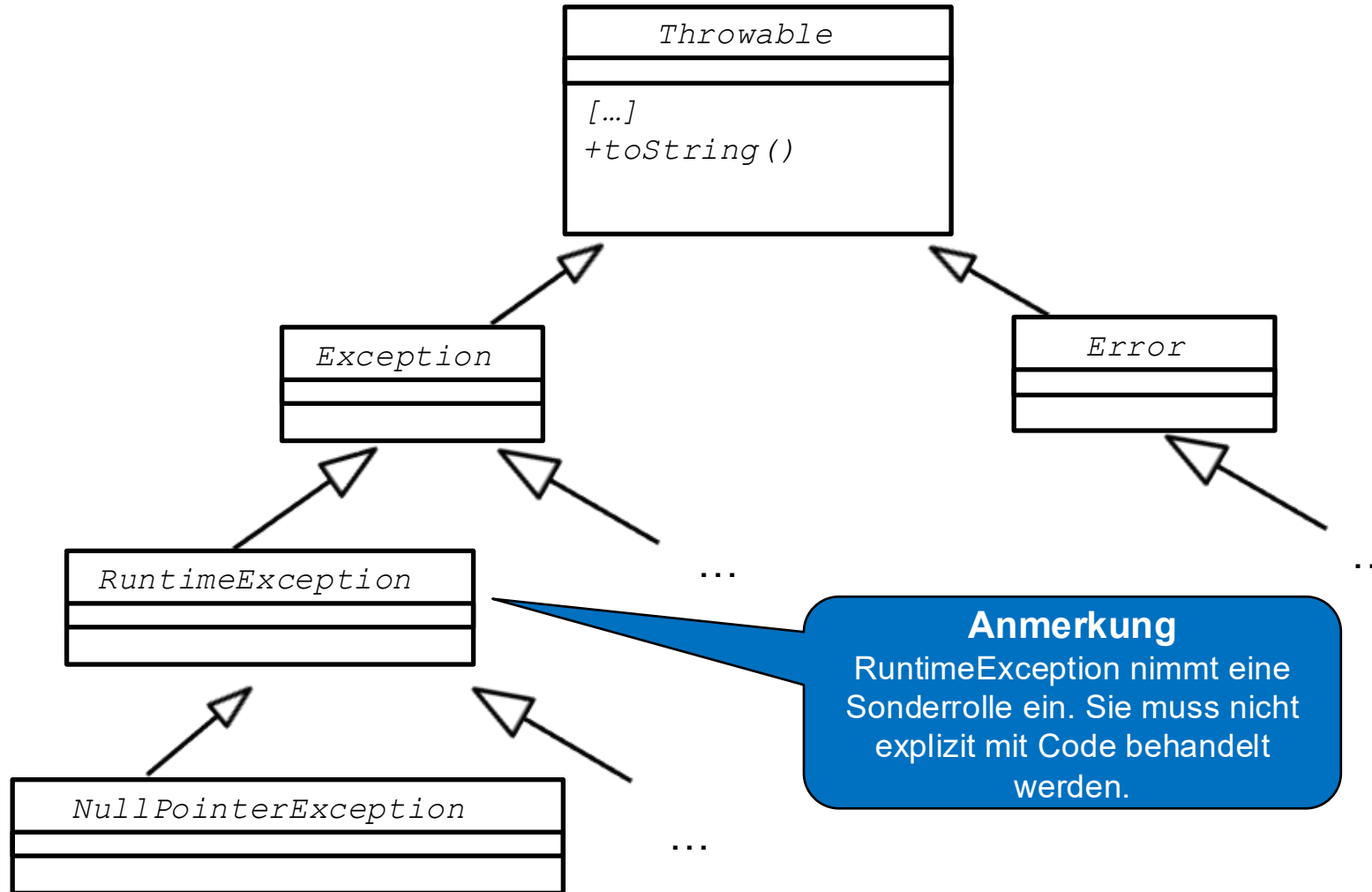
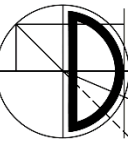
All Implemented Interfaces:

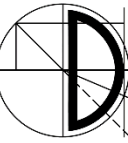
[Serializable](#)

Direct Known Subclasses:

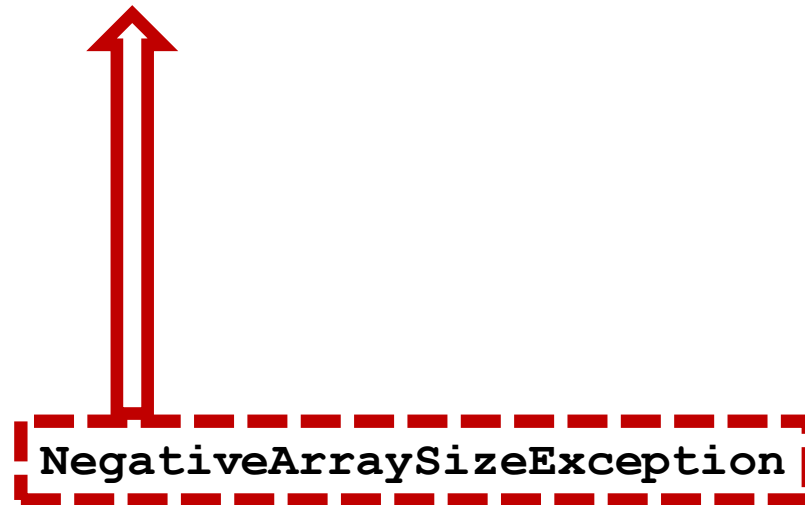
[AccessDeniedException](#), [ActivationException](#), [AlreadyBoundException](#), [ApplicationException](#), [AWTException](#), [BackingStoreException](#), [BadAttributeValueExpException](#), [BadBinaryOpValueExpException](#), [BadLocationException](#), [BadStringOperationException](#), [BrokenBarrierException](#), [CertificateException](#), [ClassNotFoundException](#), [CloneNotSupportedException](#), [DataFormatException](#), [DatatypeConfigurationException](#), [DestroyFailedException](#), [ExecutionException](#), [ExpandVetoException](#), [FontFormatException](#), [GeneralSecurityException](#), [GSSEException](#), [IllegalAccessException](#), [IllegalClassFormatException](#), [InstantiationException](#), [InterruptedException](#), [IntrospectionException](#), [InvalidApplicationException](#), [InvalidMidiDataException](#), [InvalidPreferencesFormatException](#), [InvalidTargetObjectTypeException](#), [InvocationTargetException](#), [IOException](#), [JAXBException](#), [JMEException](#), [KeySelectorException](#), [LastOwnerException](#), [LineUnavailableException](#), [MarshalException](#), [MidiUnavailableException](#), [MimeTypeParseException](#), [MimeTypeParseException](#), [NamingException](#), [NoninvertibleTransformException](#), [NoSuchFieldException](#), [NoSuchMethodException](#), [NotBoundException](#), [NotOwnerException](#), [ParseException](#), [ParserConfigurationException](#), [PrinterException](#), [PrintException](#), [PrivilegedActionException](#), [PropertyVetoException](#), [RefreshFailedException](#), [RemarshalException](#), [RuntimeException](#), [SAXException](#), [ScriptException](#), [ServerNotActiveException](#), [SOAPException](#), [SQLException](#), [TimeoutException](#), [TooManyListenersException](#), [TransformerException](#), [TransformException](#), [UnmodifiableClassException](#), [UnsupportedAudioFileException](#), [UnsupportedCallbackException](#), [UnsupportedFlavorException](#), [UnsupportedLookAndFeelException](#), [URIReferenceException](#), [URISyntaxException](#), [UserException](#), [XAException](#), [XMLParseException](#), [XMLSignatureException](#), [XMLStreamException](#), [XPathException](#)

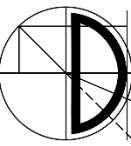
Quelle: <http://docs.oracle.com/javase/6/docs/api/java/lang/Exception.html>



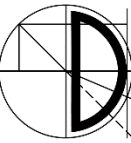


```
int[] zfeld;  
Scanner in = new Scanner(System.in);  
System.out.print("Wie lang soll das Feld sein? ");  
zfeld = new int[in.nextInt()];  
.  
.  
.
```

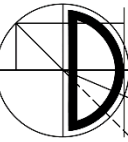




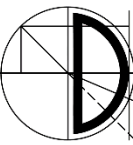
```
try
{
    // In diesem Teil können Exceptions auftreten
}
catch(Exception e)
{
    // Hier werden sie abgearbeitet
    // catch ist optional, wenn ein finally-Block folgt.
    // Es können mehrere catch-Blöcke hintereinander stehen, um
    // unterschiedliche Exceptions abzufangen!
}
finally
{
    // Wird immer durchlaufen, ist allerdings optional
}
```



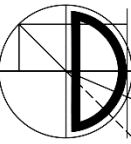
```
int[] zfeld;  
Scanner in = new Scanner(System.in);  
System.out.print("Wie lang soll das Feld sein? ");  
try {  
    zfeld = new int[in.nextInt()];  
} catch (NegativeArraySizeException e) {  
    System.out.print("Bitte nur positive Werte! Wie lang soll das Feld sein? ");  
    zfeld = new int[in.nextInt()];  
}  
.  
.  
.
```



- Wie können mir Exceptions nützlich sein?
 - Mit Hilfe von Exceptions kann ich Ausnahmen abfangen, bei denen die Wahrscheinlichkeit besteht, dass sie auftreten. Das ist besonders hilfreich, wenn ich vom Nutzer/der Nutzerin Eingaben benötige und nicht validieren kann, ob diese geeignet sind. Das Programm muss dann nicht automatisch beendet werden, sondern kann ggf. korrigiert weiterlaufen.
- Warum ist die Vererbungshierarchie von Wichtigkeit?
- Treten Exceptions nur zur Laufzeit auf?



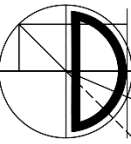
```
int[] zfeld;
Scanner in = new Scanner(System.in);
System.out.print("Wie lang soll das Feld sein? ");
try {
    zfeld = new int[in.nextInt()];
    for (int i = 0; i < zfeld.length; i++) {
        zfeld[i] = i * i;
    }
    System.out.print(" Welches El. soll ausgegeben werden? ");
    System.out.println(zfeld[in.nextInt()]);
} catch (NegativeArraySizeException nase) {
    System.out.print("Nur positive Werte erlaubt.");
} catch (Exception e) {
    System.out.print("Eine Ausnahme ist aufgetreten.");
} finally {
    System.out.println(" Programm wird beendet...");
}
```



```
int[] zfeld;  
Scanner in = new Scanner(System.in);  
System.out.print("Wie lang soll das Feld sein? ");  
try {  
    zfeld = new int[in.nextInt()];  
    for (int i = 0; i < zfeld.length; i++) {  
        zfeld[i] = i * i;  
    }  
    System.out.print(" Welches El. soll ausgegeben werden? ");  
    System.out.println(zfeld[in.nextInt()]);  
} catch (NegativeArraySizeException nase) {  
    System.out.print("Nur positive Werte erlaubt.");  
} catch (Exception e) {  
    System.out.print("Eine Ausnahme ist aufgetreten.");  
} finally {  
    System.out.println(" Programm wird beendet...");  
}
```

NegativeArraySizeException

ArrayIndexOutOfBoundsException



```
int[] zfeld;  
Scanner in = new Scanner(System.in);  
System.out.print("Wie lang soll das Feld sein? ");  
try {  
    zfeld = new int[in.nextInt()];  
    for (int i = 0; i < zfeld.length; i++) {  
        zfeld[i] = i * i;  
    }  
    System.out.print(" Welches El. soll ausgegeben werden? ");  
    System.out.println(zfeld[in.nextInt()]);  
} catch (NegativeArraySizeException nase) {  
    System.out.print("Nur positive Werte erlaubt.");  
} catch (Exception e) {  
    System.out.print("Eine Ausnahme ist aufgetreten.");  
} finally {  
    System.out.println(" Programm wird beendet...");  
}
```

Potentieller Fehler 0

Negative Eingabe => NegativeArraySizeException

Potentieller Fehler 1

Negative/zu Große Eingabe =>
ArrayIndexOutOfBoundsException

Catch Block 0

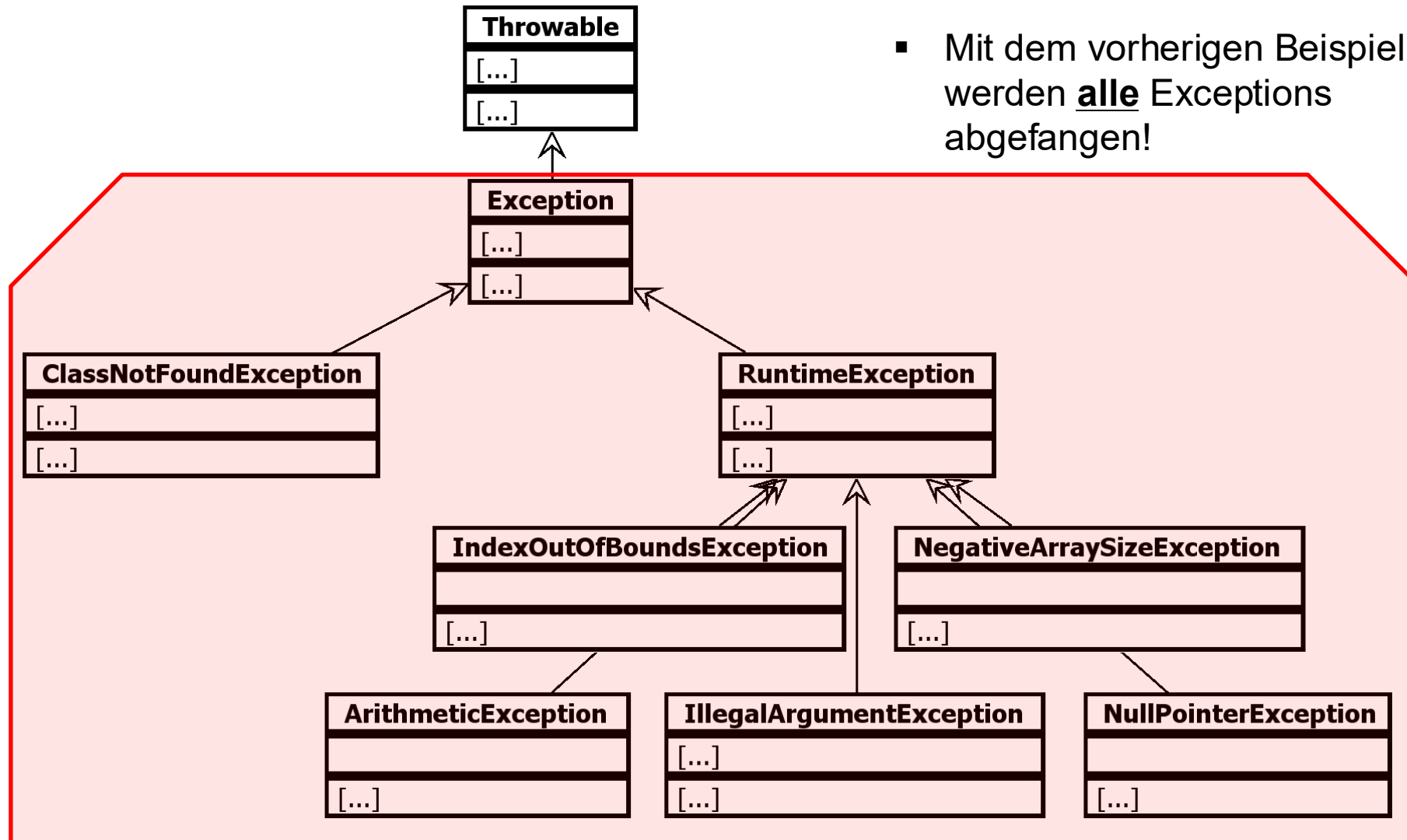
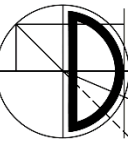
Wird zuerst überprüft. Eine Exception, die hier
gefangen wird, löst späteres Catch nicht aus.

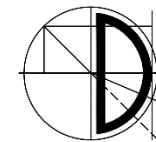
Catch Block 1

Wird danach überprüft. Hier eine allgemeinere
Exception, die u.a.
ArrayIndexOutOfBoundsException mit fängt.

Finally

Wird immer am Ende ausgeführt. Kann auch ohne Exceptions
nötig sein, um Verhalten am Ende zu garantieren.
Beispiel: Geöffnete Dateien schließen.

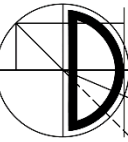




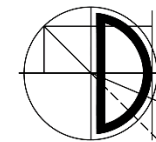
```
int[] zfeld;  
Scanner in = new Scanner(System.in);  
System.out.print("Wie lang soll das Feld sein? ");  
try {  
    zfeld = new int[in.nextInt()];  
    for (int i = 0; i < zfeld.length; i++) {  
        zfeld[i] = i * i;  
    }  
    System.out.print(" Welches El. soll ausgegeben werden? ");  
    System.out.println(zfeld[in.nextInt()]);  
} catch (NegativeArraySizeException nase) {  
    System.out.print("Nur positive Werte erlaubt.");  
} catch (Exception e) {  
    System.out.print("Eine Ausnahme ist aufgetreten.");  
} finally {  
    System.out.println(" Bitte beim nächsten Programmstart beachten. Programm wird beendet...");  
}
```

Ungeeignet!

- Zu unspezifisch
- Keine angemessene Ausnahmebehandlung



```
int[] zfeld;
Scanner in = new Scanner(System.in);
System.out.print("Wie lang soll das Feld sein? ");
try {
    zfeld = new int[in.nextInt()];
    for (int i = 0; i < zfeld.length; i++) {
        zfeld[i] = i * i;
    }
    System.out.print(" Welches El. soll ausgegeben werden? ");
    System.out.println(zfeld[in.nextInt()]);
} catch (NegativeArraySizeException nase) {
    System.out.print("Nur positive Werte erlaubt.");
} catch (ArrayIndexOutOfBoundsException aioobe) {
    System.out.print("Nur Komponenten innerhalb des Feldes erlaubt.");
} finally {
    System.out.println(" Bitte beim nächsten Programmstart beachten. Programm wird beendet...");
}
```



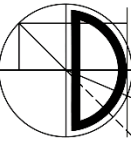
```
int[] zfeld;
Scanner in = new Scanner(System.in);
System.out.print("Wie lang soll das Feld sein? ");
try {
    zfeld = new int[in.nextInt()];
    for (int i = 0; i < zfeld.length; i++) {
        zfeld[i] = i * i;
    }
    System.out.print(" Welches El. soll ausgegeben werden? ");
    System.out.println(zfeld[in.nextInt()]);
} catch (NegativeArraySizeException nase) {
    System.out.print("Nur positive Werte erlaubt.");
} catch (ArrayIndexOutOfBoundsException aioobe) {
    System.out.print("Nur Komponenten innerhalb des Feldes erlaubt.");
} finally {
    System.out.println(" Bitte beim nächsten Programmstart beachten. Programm wird beendet...");
}
```

Programm wird ausgeführt ...

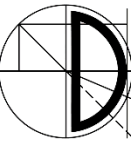
Wie lang soll das Feld sein? -1

Nur positive Werte erlaubt. Bitte beim nächsten Programmstart beachten. Programm wird beendet...

Programm beendet



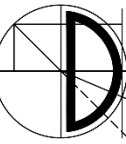
```
int[] zfeld;
Scanner in = new Scanner(System.in);
System.out.print("Wie lang soll das Feld sein? ");
try {
    zfeld = new int[in.nextInt()];
    for (int i = 0; i < zfeld.length; i++) {
        zfeld[i] = i * i;
    }
    System.out.print(" Welches El. soll ausgegeben werden? ");
    System.out.println(zfeld[in.nextInt()]);
} catch (NegativeArraySizeException nase) {
    System.out.print("Nur positive Werte erlaubt.");
} catch (ArrayIndexOutOfBoundsException aioobe) {
    System.out.print("Nur Komponenten innerhalb des Feldes erlaubt.");
} finally {
    System.out.println(" Bitte beim nächsten Programmstart beachten. Programm wird beendet...");
}
```



```
int[] zfeld;
Scanner in = new Scanner(System.in);
System.out.print("Wie lang soll das Feld sein? ");
try {
    zfeld = new int[in.nextInt()];
    for (int i = 0; i < zfeld.length; i++) {
        zfeld[i] = i * i;
    }
    System.out.print(" Welches El. soll ausgegeben werden? ");
    System.out.println(zfeld[in.nextInt()]);
} catch (NegativeArraySizeException nase) {
    System.out.print("Nur positive Werte erlaubt.");
} catch (ArrayIndexOutOfBoundsException aioobe) {
    System.out.print("Nur Komponenten innerhalb des Feldes erlaubt.");
} finally {
    System.out.println(" Bitte beim nächsten Programmstart beachten. Programm wird beendet...");
}
```

```
Programm wird ausgeführt ...
Wie lang soll das Feld sein? 10
Welches Element soll ausgegeben werden? 10
Nur Komponenten innerhalb des Feldes erlaubt. Bitte beim nächsten Programmstart beachten. Programm wird beendet...

Programm beendet
```



- **Exceptions** helfen beim Aufdecken von Lücken und Problemen im eigenen Programm
- **Exceptions** können durch den Nutzer/die Nutzerin abgefangen werden (catching)
- Beim Abfangen ist die Vererbungslinie zu beachten, um die spezifisch-gewollten **Exceptions** geeignet abzufangen

Schon gewusst, dass viele neue Sprachen keine Runtime-Exceptions mehr haben?

Als Beispiele für moderne Sprachen ohne Runtime Exceptions können z.B. Go und Rust genannt werden. In Go werden Fehler anhand von Rückgabewerten bestimmt. In Rust werden zur Compile-Zeit bereits deutlich mehr Fehler abgefangen. Der Rust-Compiler besitzt dafür viele fortgeschrittene Funktionen. Übrigens: Die Programmiersprachen Go und Rust behandeln wir auch in der Vorlesung „Advanced Programming“.



"It's the latest innovation in office safety. When your computer crashes, an air bag is activated so you won't bang your head in frustration."

The image features a background of a classical building facade with columns and arches, likely the main building of the University of Würzburg. On the left, there is a blue-tinted area with tree branches. Overlaid on this is a white logo consisting of a stylized 'J' and 'M' forming a square. To the right of the logo, the text 'Julius-Maximilians-' is in a smaller font, and 'UNIVERSITÄT WÜRZBURG' is in a larger, bold, sans-serif font.

Julius-Maximilians-
**UNIVERSITÄT
WÜRZBURG**

Fragen?

Vielen Dank für Ihre Aufmerksamkeit